

Dùng tính đơn điệu để tối ưu quy hoạch động

JSOI 2009 training team paper

Nguồn gốc: *Optimizing Dynamic Programming with Monotonicity*

Dynamic Programming / Monotonicity-optimized Dynamic Programming.pdf

Tóm tắt

Tính đơn điệu là một loại tính chất rất quan trọng. Trong thi tin học, nó là một điểm đột phá thường gặp khi tìm lời giải, đồng thời giữ vai trò then chốt trong quá trình tối ưu quy hoạch động. Bài viết này chọn một số đề trong các kỳ thi trong nước, qua đó thảo luận các ứng dụng tinh tế của tính đơn điệu trong tối ưu quy hoạch động.

1 Hàng đợi đơn điệu là gì?

Hàng đợi đơn điệu, đúng như tên gọi, là một hàng đợi mà các phần tử trong đó giữ thứ tự đơn điệu. Vì vậy phần tử ở đầu hàng đợi luôn là phần tử nhỏ nhất hoặc lớn nhất, phù hợp với nhu cầu lấy giá trị tối ưu trong quy hoạch động.

Nói chính xác hơn, đây là một hàng đợi hai đầu. Khác với hàng đợi thông thường, vốn chỉ xóa ở đầu và chèn ở cuối, hàng đợi hai đầu cho phép xóa ở cả đầu lẫn cuối.

Tính chất

Trong các bài quy hoạch động, mỗi phần tử trong hàng đợi đơn điệu thường lưu hai giá trị:

$$(\text{pos}, \text{val}),$$

trong đó pos là vị trí trong dãy ban đầu, còn val là giá trị trạng thái hoặc một giá trị được suy ra từ trạng thái. Hàng đợi đơn điệu bảo đảm cả hai đại lượng này đều đơn điệu theo thứ tự phần tử trong hàng đợi.

Dùng để làm gì?

Xét bài toán sau: cho dãy có n phần tử, với $n \leq 2\,000\,000$. Với mỗi vị trí i , hãy tìm giá trị nhỏ nhất trong đoạn gồm m phần tử kết thúc tại i . Nếu ký hiệu phần tử thứ i là $a[i]$, đáp án là

$$f(i) = \min_{j=i-m+1}^i a[j].$$

Cách làm trực tiếp có độ phức tạp $O(nm)$; các cấu trúc RMQ kiểu cây đoạn hoặc bảng thưa cũng vẫn có hằng số hoặc bậc logarit không cần thiết cho bài này.

Ta duy trì một hàng đợi gồm các cặp (pos, val) , trong đó pos là chỉ số của một phần tử trong dãy và $\text{val} = a[\text{pos}]$. Các vị trí trong hàng đợi tăng dần, còn các giá trị cũng tăng dần. Khi tính $f(i)$, ta xóa các

phần tử ở đầu hàng đợi cho tới khi $\text{pos} \geq i - m + 1$. Khi đó phần tử đầu hàng đợi chính là giá trị nhỏ nhất trong cửa sổ hiện tại.

Khi cần chèn $a[i]$ vào cuối hàng đợi, thứ tự vị trí tự động tăng vì ta quét i từ trái sang phải. Để giữ giá trị tăng dần, ta liên tục xóa các phần tử cuối có giá trị lớn hơn $a[i]$, rồi mới chèn i .

Mỗi phần tử được chèn nhiều nhất một lần và bị xóa nhiều nhất một lần, nên tổng thời gian là $O(n)$.

Vì sao thao tác này đúng?

Giả sử $j < i$ và $a[j] > a[i]$. Từ thời điểm i trở đi, mọi cửa sổ còn chứa j mà có thể dùng j làm ứng viên thì cũng chứa i hoặc sẽ chứa i lâu hơn j , vì biên trái $i - m + 1$ của cửa sổ tăng đơn điệu theo i . Do đó j không thể tốt hơn i trong bất kỳ trạng thái tương lai nào. Xóa j ở cuối hàng đợi là hợp lý.

Tương tự, ta có thể xóa ở đầu hàng đợi vì biên trái của đoạn hợp lệ cũng tăng đơn điệu. Một phần tử một khi đã nằm ngoài cửa sổ thì sẽ không bao giờ quay lại.

Mẫu quy hoạch động tương ứng

Một lớp bài toán thường dùng được hàng đợi đơn điệu có dạng

$$f(x) = \text{opt}_{i=\text{bound}[x]}^{x-1} \text{const}[i],$$

trong đó $\text{bound}[x]$ không giảm theo x , còn $\text{const}[i]$ là một hằng số xác định duy nhất từ i và tính được trong $O(1)$. Khi đó hàng đợi đơn điệu loại bỏ việc liệt kê toàn bộ các quyết định.

2 Một vài ví dụ đơn giản

2.1 Ví dụ 1: Sản xuất sản phẩm, Vijos 1243

Đề bài

Có n sản phẩm, đánh số từ 1 đến n , cần được sản xuất theo đúng thứ tự tăng dần. Có m robot. Thời gian để robot j sản xuất sản phẩm i là $T[i, j]$. Cùng một robot không được sản xuất liên tiếp quá l sản phẩm. Hỏi thời gian ít nhất để hoàn thành toàn bộ sản phẩm. Ràng buộc:

$$n \leq 10^5, \quad m \leq 5, \quad l \leq 5 \cdot 10^4.$$

Phân tích

Gọi $f[i, j]$ là thời gian nhỏ nhất để sản xuất xong i sản phẩm đầu, trong đó sản phẩm thứ i được hoàn thành bởi robot j . Đặt $\text{sum}[x, y]$ là tổng thời gian để robot y sản xuất các sản phẩm từ 1 đến x . Nếu đoạn cuối do robot j sản xuất bắt đầu sau vị trí k , và sản phẩm k do robot $p \neq j$ sản xuất, ta có

$$f[i, j] = \min_{k=i-l}^{i-1} \min_{p \neq j} \{f[k, p] + \text{sum}[i, j] - \text{sum}[k, j]\}.$$

Cách tính trực tiếp mất $O(nm^2l)$.

Vì m rất nhỏ, ta xét từng robot j riêng. Với j cố định,

$$f[i, j] = \min_{k=i-l}^{i-1} \left\{ \min_{p \neq j} f[k, p] - \text{sum}[k, j] \right\} + \text{sum}[i, j].$$

Biểu thức trong ngoặc chỉ phụ thuộc vào k , còn đoạn quyết định hợp lệ $[i - l, i - 1]$ trượt đơn điệu khi i tăng. Do đó với mỗi j ta duy trì một hàng đợi đơn điệu chứa giá trị

$$\min_{p \neq j} f[k, p] - \text{sum}[k, j].$$

Như vậy một chiều liệt kê quyết định được loại bỏ, và độ phức tạp giảm còn $O(nm^2)$.

2.2 Ví dụ 2: Cut the Sequence, PKU 3017

Đề bài

Cho dãy a gồm n số nguyên không âm. Cần chia dãy thành một số đoạn sao cho tổng của mỗi đoạn không vượt quá hằng số M , đồng thời tổng các giá trị lớn nhất của từng đoạn là nhỏ nhất. Ràng buộc $n \leq 10^5$.

Ví dụ với $n = 8, M = 17$ và dãy

$$2, 2, 2, 8, 1, 8, 1, 2,$$

một cách chia tối ưu là

$$2, 2, 2 \mid 8, 1, 8 \mid 1, 2,$$

có đáp án 12.

Phân tích

Gọi $f(i)$ là chi phí nhỏ nhất để chia i phần tử đầu. Đặt

$$b[i] = \min\{j \mid \text{sum}(j + 1, i) \leq M\},$$

có thể tính bằng hai con trỏ hoặc tìm kiếm nhị phân trên tổng tiền tố. Khi đó

$$f(i) = \min_{j=b[i]}^{i-1} \{f(j) + \max_{t=j+1}^i a[t]\}.$$

Nếu M rất lớn, cách trực tiếp có thể mất $O(n^2)$.

Ta quan sát ba tính chất.

1. Nếu quyết định k được dùng khi tính $f(x)$, thì phần tử k và phần tử x không nằm trong cùng một đoạn.
2. $b[x]$ không giảm theo x , đúng với điều kiện cần của hàng đợi đơn điệu.
3. Một quyết định k có thể là quyết định tối ưu cho trạng thái $f(x)$ chỉ khi

$$a[k] > a[j] \quad (k < j \leq x).$$

Nếu sau k còn một phần tử không nhỏ hơn $a[k]$, việc cắt tại k không đem lại giá trị lớn nhất tốt hơn nhưng lại không có lợi về f .

Vì vậy với mỗi x , tập quyết định hữu ích tạo thành một dãy có giá trị $a[\cdot]$ giảm dần. Ta có thể dùng cơ chế giống hàng đợi đơn điệu: xóa các quyết định đã nằm trước $b[x]$, rồi chèn x vào cuối và duy trì dãy giảm theo a .

Tuy nhiên đầu hàng đợi không nhất thiết là quyết định tối ưu, vì nó chỉ bảo đảm phần tử đại diện có giá trị lớn nhất. Với các phần tử trong hàng đợi, trừ phần tử cuối, giả sử phần tử thứ r có vị trí $q[r]$, giá trị ứng viên do nó sinh ra là

$$t_r = f(q[r]) + a[q[r + 1]].$$

Mỗi lần xóa ở đầu hoặc cuối tương ứng với việc xóa một giá trị t_r ; sau đó có thể cần chèn một giá trị t_r mới. Ta cần truy vấn giá trị nhỏ nhất trong tập các t_r . Một cây cân bằng hoặc multiset xử lý được việc chèn, xóa và lấy nhỏ nhất trong $O(\log n)$.

Cần chú ý riêng quyết định $b[x]$: nó có thể bị cơ chế hàng đợi xóa mất, nhưng vẫn là một điểm cắt hợp lệ cần xét. Vì vậy khi tính $f(x)$, ta so sánh thêm ứng viên

$$f(b[x]) + \max_{t=b[x]+1}^x a[t].$$

Tổng độ phức tạp là $O(n \log n)$.

Kết luận nhỏ

Trong ví dụ này ta vẫn dựa vào tính đơn điệu, nhưng giá trị cần duy trì không phải chỉ là phần tử ở đầu hàng đợi. Hàng đợi đơn điệu là một công cụ linh hoạt; tùy bài, nó có thể cần phối hợp với một cấu trúc dữ liệu khác.

3 Một vài ví dụ phức tạp hơn

3.1 Ví dụ 3: Đóng gói đồ chơi, HNOI 2008

Đề bài

Có n món đồ chơi, cần chia chúng thành các nhóm liên tiếp để đóng gói. Món thứ i có độ dài $\text{len}[i]$. Nếu đóng gói các món từ x đến y vào cùng một nhóm, độ dài của nhóm là

$$l = y - x + \sum_{i=x}^y \text{len}[i],$$

và chi phí của nhóm là $(l - L)^2$. Số món trong một nhóm không bị giới hạn. Hãy tìm chi phí nhỏ nhất để đóng gói toàn bộ đồ chơi, với $n \leq 5 \cdot 10^4$.

Cách làm trực tiếp

Gọi $f(x)$ là chi phí nhỏ nhất để đóng gói x món đầu. Nếu $w[i, x]$ là chi phí đóng gói các món từ $i + 1$ đến x vào một nhóm, thì

$$f(x) = \min_{0 \leq i < x} \{f(i) + w[i, x]\}.$$

Độ phức tạp trực tiếp là $O(n^2)$.

Tối ưu bằng tính đơn điệu quyết định

Nếu in quyết định tối ưu $k[x]$ của từng trạng thái, ta sẽ thấy

$$i < j \implies k[i] \leq k[j].$$

Tức là quyết định tối ưu không lùi về trái khi trạng thái tăng.

Tạm thời chưa xét chứng minh, ta dùng tính chất này để tối ưu. Ban đầu $f[0] = 0$; trước khi có trạng thái nào khác, mọi trạng thái tương lai chỉ có quyết định 0. Quét i từ 1 đến n . Khi tới i , ta bảo đảm quyết định của $f[i]$ đã biết và có thể tính $f[i]$.

Sau khi tính xong $f[i]$, ta tìm xem từ trạng thái nào trở đi thì quyết định i tốt hơn các quyết định cũ. Do tính đơn điệu, nếu trong các quyết định $0, \dots, i$, quyết định tốt nhất của $f[x]$ là i , thì với mọi $y > x$, quyết định tốt nhất trong cùng tập quyết định đó cũng là i . Vì vậy có thể tìm điểm chuyển bằng tìm kiếm nhị phân.

Nếu dùng cây đoạn để tô đoạn quyết định $[x, n]$, đồng thời truy vấn quyết định hiện tại của từng trạng thái, độ phức tạp là $O(n \log^2 n)$. Trong thực tế, cách gọn hơn và hằng số nhỏ hơn là duy trì một ngăn xếp các đoạn liên tiếp, mỗi đoạn lưu một quyết định tối ưu chung. Khi một quyết định mới xuất hiện, ta xóa các đoạn mà nó hoàn toàn tốt hơn, rồi tìm điểm cắt trong đoạn biên bằng tìm kiếm nhị phân.

Vì sao quyết định đơn điệu?

Với những bài quy hoạch động có $O(n)$ trạng thái và $O(n)$ quyết định, nếu chi phí chuyển từ trạng thái x sang y là $w[x, y]$ và thỏa bất đẳng thức tứ giác

$$w[x, y] + w[x + 1, y + 1] \leq w[x + 1, y] + w[x, y + 1],$$

thì quyết định tối ưu là đơn điệu. Bài đóng gói đồ chơi thỏa điều kiện này sau khi viết w theo tổng tiền tố của độ dài đồ chơi; từ đó suy ra $k[i] \leq k[j]$ với $i < j$. Bài viết gốc chỉ nêu hướng chứng minh và khuyên đọc thêm tài liệu về bất đẳng thức tứ giác.

3.2 Ví dụ 4: Xây kho, ZJTSC 2007

Đề bài

Có n điểm bán hàng nằm lần lượt từ đỉnh núi xuống chân núi. Điểm 1 ở cao nhất, điểm n ở chân núi. Vì sắp có mưa lớn, cần chọn một số điểm để xây kho. Hàng hóa chỉ được vận chuyển xuống núi. Vận chuyển một đơn vị hàng qua một đơn vị khoảng cách tốn 1 đồng.

Với mỗi điểm i , biết khoảng cách $x[i]$ từ điểm 1 đến điểm i , lượng hàng $p[i]$, và chi phí xây kho $c[i]$. Hãy chọn phương án có tổng chi phí vận chuyển và xây kho nhỏ nhất. Ràng buộc $n \leq 10^6$.

Phân tích

Gọi $f[i]$ là tổng chi phí nhỏ nhất để xử lý các điểm $1, \dots, i$, với một kho được xây ở điểm i . Khi kho trước đó ở sau điểm j , chi phí vận chuyển các điểm $j + 1, \dots, i$ về kho i là $w[j, i]$, do đó

$$f(i) = \min_{0 \leq j < i} \{f[j] + w[j, i]\} + c[i].$$

Ta có

$$\begin{aligned} w[j, i] &= p[j + 1](x[i] - x[j + 1]) + \dots + p[i](x[i] - x[i]) \\ &= x[i](p[j + 1] + \dots + p[i]) - (p[j + 1]x[j + 1] + \dots + p[i]x[i]). \end{aligned}$$

Đặt

$$\text{sump}[t] = \sum_{r=1}^t p[r], \quad \text{sum}[t] = \sum_{r=1}^t p[r]x[r].$$

Khi đó

$$w[j, i] = -x[i] \text{sump}[j] + \text{sum}[j] + x[i] \text{sump}[i] - \text{sum}[i].$$

Thay vào công thức quy hoạch động:

$$f(i) = \min_{0 \leq j < i} \{f[j] + \text{sum}[j] - x[i] \text{sump}[j]\} + x[i] \text{sump}[i] - \text{sum}[i] + c[i].$$

Phần sau dấu ngoặc chỉ phụ thuộc vào i nên không ảnh hưởng tới quyết định.

Đặt

$$a[i] = -x[i], \quad X(j) = \text{sump}[j], \quad Y(j) = f[j] + \text{sum}[j].$$

Ta cần tính

$$\min_{0 \leq j < i} \{a[i]X(j) + Y(j)\}.$$

Nếu gọi giá trị cần tối ưu là G , ta có đường thẳng

$$G = aX + Y \iff Y = -aX + G.$$

Với hệ số góc đã biết, chọn điểm $(X(j), Y(j))$ sao cho G nhỏ nhất nghĩa là tịnh tiến một đường thẳng cho tới khi chạm tập điểm; điểm chạm đầu tiên nằm trên bao lồi dưới.

Trong bài này, hệ số góc thay đổi đơn điệu khi i tăng, và hoành độ $X(j) = \text{sump}[j]$ cũng tăng đơn điệu. Vì vậy khi quét các trạng thái, điểm quyết định tối ưu trên bao lồi chỉ di chuyển sang phải. Đây cũng là một cách nhìn khác để thấy quyết định đơn điệu.

Thuật toán

Duy trì một ngăn xếp các điểm trên bao lồi dưới. Ngoài con trỏ đỉnh ngăn xếp, ta giữ thêm con trỏ quyết định t , chỉ vị trí bắt đầu tìm quyết định tối ưu hiện tại. Nhờ tính đơn điệu, nếu một điểm không còn là quyết định tối ưu ở trạng thái hiện tại, nó cũng không thể trở thành tối ưu ở các trạng thái sau; vì vậy t chỉ tăng.

Với trạng thái i , đặt

$$V_t = f[t] + w[t, i].$$

Ta dời con trỏ bằng quy tắc

$$\text{while } V_t \geq V_{t+1} \text{ do } t \leftarrow t + 1.$$

Khi dừng, t là quyết định tốt nhất để tính $f[i]$.

Sau khi tính xong $f[i]$, ta chèn điểm mới

$$(\text{sump}[i], f[i] + \text{sum}[i])$$

vào bao lồi. Cách chèn giống bước duy trì trong Graham scan: liên tục xóa đỉnh cuối nếu ba điểm cuối không còn tạo thành bao lồi dưới, rồi chèn điểm mới. Nếu con trỏ quyết định bị xóa khỏi ngăn xếp, đặt nó về điểm hiện tại phù hợp với tính đơn điệu.

Vòng lặp chính chạy n lần; con trỏ quyết định chỉ tăng tổng cộng $O(n)$ lần; mỗi điểm vào và ra ngăn xếp nhiều nhất một lần. Do đó độ phức tạp là $O(n)$.

4 Dùng tính đơn điệu của đường lồi để tối ưu quy hoạch động

Ví dụ xây kho dùng phép biến đổi đại số để đưa bài toán về tối ưu một hàm tuyến tính trên tập điểm. Trường hợp đó thuận lợi vì hệ số góc của đường thẳng thay đổi đơn điệu và hoành độ của các điểm cũng đơn điệu. Nếu một trong hai điều kiện này không còn đúng, ta cần một cách duy trì bao lồi tổng quát hơn.

4.1 Ví dụ 5: Đổi tiền, NOI 2007

Đề bài

Trong n ngày, Y muốn giao dịch hai loại chứng khoán A và B . Ngày thứ i , giá một đơn vị A là A_i , giá một đơn vị B là B_i , và khi mua, tỉ lệ số lượng nhận được phải thỏa

$$\frac{\text{số đơn vị } A}{\text{số đơn vị } B} = \text{Rate}_i.$$

Ban đầu Y có S tiền. Mỗi ngày có thể làm một trong ba việc:

1. dùng toàn bộ tiền để mua hai loại chứng khoán theo tỉ lệ của ngày đó;
2. bán toàn bộ chứng khoán đang giữ để nhận tiền theo giá của ngày đó;
3. không làm gì.

Hỏi sau n ngày có thể có nhiều nhất bao nhiêu tiền. Ràng buộc $n \leq 10^5$.

Phân tích

Gọi $f(i)$ là số tiền lớn nhất có thể nhận được ở cuối ngày i nếu ngày i thực hiện thao tác bán. Sau khi biết $f(i)$, ta tính được $x(i)$ và $y(i)$: số lượng chứng khoán A và B có thể mua bằng $f(i)$ tiền theo tỉ lệ Rate_i . Các giá trị này thu được bằng cách giải hệ

$$A_i x(i) + B_i y(i) = f(i), \quad \frac{x(i)}{y(i)} = \text{Rate}_i.$$

Khi bán ở ngày i , ta chọn một ngày mua trước đó j và nhận

$$A_i x(j) + B_i y(j).$$

Do đó

$$f(i) = \max_{0 \leq j < i} \{A_i x(j) + B_i y(j)\},$$

và đáp án cuối cùng là

$$\max_{0 \leq i \leq n} f(i),$$

trong đó trạng thái ban đầu tương ứng với số tiền S .

Phương trình trên trực tiếp là $O(n^2)$. Viết

$$G = Ax + By \iff y = -\frac{A}{B}x + \frac{G}{B}.$$

Đây vẫn là bài toán chọn một điểm trên bao lồi để tối ưu đường thẳng. Khác với ví dụ xây kho, hệ số góc $-A_i/B_i$ không đơn điệu, và hoành độ $x(i)$ cũng không đơn điệu theo thời gian. Vì vậy không thể dùng một hàng đợi hoặc một con trỏ đơn điệu đơn giản.

Truy vấn trên bao lồi

Ta duy trì bao lồi trên của các điểm $(x(j), y(j))$, vì cần cực đại hóa. Với một đường thẳng truy vấn có hệ số góc

$$k = -\frac{A_i}{B_i},$$

nếu một điểm P trên bao lồi là tối ưu, gọi k_1 là hệ số góc cạnh nối từ điểm bên trái tới P , và k_2 là hệ số góc cạnh nối từ P tới điểm bên phải. Khi đó phải có

$$k_2 \leq k \leq k_1.$$

Ta có thể xem $k_1 = +\infty$ tại điểm trái nhất và $k_2 = -\infty$ tại điểm phải nhất. Vì hệ số góc các cạnh trên bao lồi được sắp thứ tự, điểm tối ưu được tìm bằng tìm kiếm nhị phân trong $O(\log n)$.

Chèn điểm

Do các điểm mới không có thứ tự hoành độ đơn điệu, cần một cây cân bằng theo hoành độ; bài viết gốc lấy splay làm ví dụ. Khi chèn điểm mới a , ta đưa nó vào cây theo hoành độ. Sau đó duy trì bao lồi ở hai phía.

Xét phía trái của a ; phía phải tương tự. Ta tìm điểm gần a nhất về bên trái còn thỏa tính chất bao lồi trên, tức dãy hệ số góc vẫn giảm. Theo đặc trưng của Graham scan, các điểm bị loại nằm thành một đoạn liên tiếp. Với splay, có thể đưa điểm biên còn giữ lại lên gốc, đưa a vào cây con phải của gốc, rồi xóa cả đoạn nằm giữa. Sau khi xử lý cả hai phía, cần kiểm tra chính a : nếu a không nằm trên bao lồi, ta xóa nó.

Nhờ cây cân bằng, mỗi lần truy vấn và chèn đều mất $O(\log n)$ theo phân tích chuẩn, nên bài toán được giải trong $O(n \log n)$.

Tổng kết

Bài viết đã dùng năm ví dụ kinh điển để trình bày vai trò của tính đơn điệu trong quy hoạch động. Các ví dụ đi từ hàng đợi đơn điệu cơ bản, qua quyết định đơn điệu, tới tối ưu bằng bao lồi và cây cân bằng. Điểm chung là luôn tìm ra một đại lượng chỉ dịch chuyển theo một hướng, hoặc một cấu trúc có thứ tự đủ mạnh để thay thế việc liệt kê toàn bộ quyết định.

Tài liệu tham khảo

1. *Introduction to 1D/1D Dynamic Programming Optimization*, Wang Yining, High School Affiliated to Nanjing Normal University.
2. *2007 China Informatics Olympiad Yearbook*, Chinese Computer Federation.
3. *The Art of Algorithms and Informatics Contests*, edited by Liu Rujia and Huang Liang.
4. *A Strengthening of Convex Complete Monotonicity and Its Applications*, China national training team paper, 2007, Yang Zhe.
5. *A Brief Discussion of Network Flow Algorithms Based on Layering Ideas*, China national training team paper, 2007, Wang Xinshang.

Lời cảm ơn

Tác giả cảm ơn Wang Yining từ High School Affiliated to Nanjing Normal University và Dong Huaxing từ Shaoxing No. 1 High School vì sự giúp đỡ lâu dài; cảm ơn Velocious Informatics Judge Online System đã cung cấp ví dụ 1; cảm ơn Peking University Online Judge đã cung cấp ví dụ 2; cảm ơn Xie Tian từ Zhenhai High School đã cung cấp đề HNOI 2008; và cảm ơn Dong Huaxing đã cung cấp ví dụ 4.